

Pixel Battle — Scripted Combat (Sub-project B)

Design Spec

Pixel Battle — Scripted Combat (Sub-project B) —

Design Spec

For agentic workers: This is a *design spec*. The step-by-step implementation plan is generated separately via `superpowers:writing-plans`. Do not implement directly from this document.

- **Date:** 2026-05-22
- **Status:** Design approved (the user approved the design and granted full autonomous execution through to completion — no further approval gates)
- **Relationship:** Sub-project **B** of the pixel_battle combat work. Follows the 2026-05-21 animation overhaul and the 2026-05-22 Sub-project A (combat-feel polish). Flash / teleport / dash movement abilities are explicitly **deferred** to a future sub-project.

1. Motivation

The pixel_battle fights are driven by a trained RL (PPO) policy. Validation has repeatedly shown the RL fights are tactically thin and uncontrollable — low action density, a “passive agent” risk, and every rendered video is a gamble on whether that match happened to be exciting. For a **video product**, emergent RL is the wrong tool: every output must be good.

The user’s decision: drive fights with **authored scripts** instead. The user also wants real skill effects (root / slow / shield / tenacity) and a mage (Lux) that keeps distance and casts — all of which a scripted approach delivers directly, with full directorial control and no retraining.

2. Goal

Replace the RL action source with a **script-driven** one: a fight is an authored, conditional sequence of intents per character, played out by the existing engine. Add a **status-effect system** (root, slow, shield, tenacity) so scripted skills have real mechanical effects. Ship a starter library of authored fight scripts that produce controlled, dramatic videos every time.

3. Scope & Constraints

In scope: 1. **ScriptDriver** — a script-based action source replacing the RL policy. 2. **Script format + loader** — conditional intent sequences as YAML data files. 3. **Status-effect system** — root, slow, shield, tenacity; data-driven via an `applies` field on skills. 4. **A starter library** — ~5 authored fight scripts. 5. **Part 0 — renderer micro-polish:** smaller characters (lower `CAM_ZOOM`) and smoother animation (pose interpolation spanning the full engine phase).

Hard constraint: No RL retraining. The ScriptDriver replaces RL as the action source; the trained checkpoint becomes unused for scripted videos. `characters.json` gains an `applies` field on skills — pure data, it does not affect checkpoint loadability and scripted videos do not use the checkpoint anyway.

Out of scope (deferred): Flash / teleport / dash movement abilities; LLM auto-generation of scripts (the format is designed to make this an easy future addition, but B authors scripts by hand).

Existing RL code stays — `play.py`'s RL render path is left intact (not removed); the scripted path is added alongside.

4. Current state (verified)

- The engine (`Battle`) runs the simulation; `tick_ms(dt_ms)` advances one tick. Physics, skills, hit resolution, hitstop, and stagger all live in the engine.
- Actions reach the engine via `env.step(left_action, right_action) → _apply_action(me, opp, action)` (`env.py`) → `battle.tick_ms()`. Actions are `Discrete(9)` integers: 0 idle, 1 retreat, 2 advance, 3 jump, 4 basic, 5 cd, 6 ultimate, 7 special, 8 kick.
- `play.py`'s `_render_fight` drives a render by stepping the env each frame, currently sourcing actions from a PPO `model`.
- There is **no** status-effect / shield / CC concept anywhere in the engine — skills only deal damage.
- `Skill` (`engine/skill.py`) is loaded from `characters.json`; it has no effect field.
- Sub-project A added engine-layer hitstop and a per-skill attack-range gate — both compatible with scripted play.

5. Architecture — ScriptDriver as the action source

The engine — physics, skills, hit resolution, status effects, hitstop, rendering, VFX, audio, camera — is **unchanged in how it runs a tick**. The only thing that changes is **where the per-tick actions come from**.

- `_render_fight` becomes **action-source-agnostic**. Its `model` parameter is generalised to an `action_source` callable: `action_source(env) → (left_action, right_action)`, called once per tick. The existing RL path passes a thin wrapper around the PPO model (builds the obs, calls `model.predict`); the scripted path passes a `ScriptDriver`.

- **ScriptDriver** holds the loaded fight script (two intent sequences). Each tick, given the live `Battle`, it: for each character, evaluates the active intent's `until` condition; advances the intent index if met; returns the action for the active intent's `do`. It reads raw `Battle / Character` state (positions, hp, action_state, effects) to evaluate conditions.
- **New scripted render entry point** — `play_scripted.py` — loads a script file, builds the env with the script's two characters, and renders via `_render_fight` with a `ScriptDriver` as the action source. Output: one mp4 per script.

6. Script format — conditional intent sequence

A fight script is a YAML file in `pixel_battle/data/scripts/`:

```
name: "Lux vs Garen – kite and root"
left: garen
right: lux
left_script:
  - {do: advance,          until: "dist<=120"}
  - {do: "attack:basic",   until: skill_done}
  - {do: "attack:basic",   until: skill_done}
  - {do: advance,          until: "dist<=110"}
  - {do: "attack:ultimate", until: skill_done}
right_script:
  - {do: retreat,          until: "dist>=230"}
  - {do: "attack:cd",      until: skill_done}    # light_binding – roots Garen
  - {do: retreat,          until: "dist>=210"}
  - {do: "attack:special", until: skill_done}    # lucent_singularity
  - {do: retreat,          until: "dist>=230"}
```

do verbs (each maps to an existing engine action):

verb	engine action
idle	0
retreat	1 (move away from opponent)
advance	2 (move toward opponent)
jump	3
attack:basic	4
attack:cd	5
attack:ultimate	6

verb	engine action
<code>attack:special</code>	7
<code>attack:kick</code>	8

`attack:<kind>` uses the engine's existing skill selection — the engine picks the character's skill of that kind. (Selecting a specific skill by id, when a character has two of a kind, is out of scope; the starter scripts are authored around `attack:<kind>`.)

until conditions (advance to the next intent when true):

condition	meaning
<code>dist>=N</code> / <code>dist<=N</code>	horizontal distance between the two fighters (px)
<code>time>=N</code>	ms elapsed inside the current intent
<code>skill_done</code>	the character started and finished an attack during this intent (back to a non-attacking state)
<code>hp<=N</code>	the character's own HP at or below N
<code>target_hp<=N</code>	the opponent's HP at or below N
<code>target_has:<effect></code>	the opponent currently has the named status effect (e.g. <code>root</code>)

Robustness rules: - Every intent has an implicit timeout: if its `until` is not met within `INTENT_MAX_MS` (e.g. 4000 ms), the driver advances anyway — a script can never hang. - When a character's intent sequence is exhausted, the character `idle` s. - The match ends on a KO (engine) or when both scripts are exhausted plus a short tail.

7. Status-effect system

Character gains an `effects` list. Each `StatusEffect` has: `kind` (`root` | `slow` | `shield` | `tenacity`), `remaining_ms`, and `magnitude` (a shield pool size, or a slow/tenacity factor). `Battle.tick_ms` decrements `remaining_ms` each tick and removes expired effects. (Hitstop freezes `tick_ms`, so effect timers correctly pause during a freeze.)

Effect behaviours:

effect	behaviour
root	the character cannot move — physics forces <code>vel_x = 0</code> while rooted, regardless of the action issued
slow	the character's movement velocity is multiplied by <code>magnitude</code> (a factor < 1)

effect	behaviour
shield	a damage-absorption pool; incoming damage depletes the pool before it reaches HP
tenacity	incoming stagger and CC durations applied to the character are multiplied by <code>magnitude</code> (a factor < 1)

Data-driven application — the `applies` field. `Skill` gains an optional `applies`, loaded from `characters.json`:

```
{"id": "light_binding", "type": "cooldown", "dmg": 5, "anim": "light_binding",
  "vfx": "bolt", "range": "special",
  "applies": {"effect": "root", "duration_ms": 1500, "magnitude": 1.0, "target":
    "opponent"}}
```

- `target: "opponent"` effects (root, slow) are applied to the **defender on a successful hit**, inside `_resolve_attack_hit`.
- `target: "self"` effects (shield, tenacity) are applied to the **caster when the attack starts** (in `_start_attack_with_kind`) — a self-buff does not need to connect.

A first pass of `applies` fields is added to `characters.json`: at minimum `light_binding` → root, plus a slow skill, a shield skill (e.g. `prismatic_barrier`), and a tenacity skill (e.g. `courage`). The exact assignment is finalised during implementation against the existing skill roster.

Renderer: a simple per-effect indicator is drawn on an affected character — e.g. shackle marks at the feet for root, a translucent ring for shield. Minimal, readable; not a major VFX effort.

8. Part 0 — renderer micro-polish

Two small renderer changes, bundled as the first part of this sub-project:

- **Smaller characters:** lower `CAM_ZOOM` in `play.py` from 1.45 toward ~1.2 (a starting value, tuned in validation) — the fighters read smaller, more stage is visible.
- **Smoother animation:** the renderer's pose interpolation currently completes a motion in the first few frames of an attack phase and then holds (the renderer's per-archetype phase durations in `poses.py` are shorter than the engine's real `ATTACK_WINDUP_MS` / `ATTACK_ACTIVE_MS` / `ATTACK_RECOVER_MS`). Make the pose interpolation span the engine's actual phase durations so each motion uses every available frame. This is a `poses.py` change; it removes the snap-then-hold and yields visibly smoother motion.

9. Error handling

- A malformed script (unknown `do` verb, unparseable `until`, missing `left / right`, unknown character id) is rejected by the loader with a clear error — a broken script never renders a broken match.
- An attack intent whose skill cannot fire (on cooldown, insufficient MP, out of range) is a no-op for that tick (the engine already behaves this way); the intent's `skill_done` will not trigger, so the intent's implicit `INTENT_MAX_MS` timeout advances the script — no hang.
- An unknown status-effect `kind` in an `applies` field is rejected at load time.
- Effect durations and shield pools never go negative (clamped at 0; expired effects removed).

10. Testing

- **Conditions:** each of the 7 `until` condition types evaluates correctly against constructed `Battle` states.
- **ScriptDriver:** intents advance when `until` is met; the correct action is emitted per `do` verb; the implicit timeout advances a stuck intent; an exhausted script idles.
- **Script loader:** a valid YAML script loads into the expected structure; each malformed-script case raises a clear error.
- **Status effects:** root forces `vel_x = 0`; slow scales movement; shield absorbs damage before HP and then HP takes the remainder; tenacity reduces an applied stagger; every effect expires after its duration.
- **applies :** a CC skill applies its effect to the defender on hit; a self-buff applies to the caster at attack start; `Skill` parses `applies` from JSON.
- **Integration:** a full scripted match (load a starter script, run it via the ScriptDriver) reaches an end state and renders without error.
- **Part 0:** `CAM_ZOOM` is the new value; pose interpolation spans the engine phase (a motion is still progressing partway through the strike phase, not already clamped).
- All pre-existing tests stay green.

11. Validation

Render every starter script via `play_scripted.py`. Review the mp4s: each fight plays the authored choreography; the mage kites and roots; root / slow / shield / tenacity read clearly; characters are smaller and motion is smoother. Tune `CAM_ZOOM`, effect durations/magnitudes, and script intents from what the renders show.

12. Future work

- Flash / teleport / dash movement abilities as additional `do` verbs (a dedicated sub-project).

- LLM auto-generation of scripts per video — the YAML intent format is deliberately designed to make this a straightforward later addition.
- Selecting a specific skill by id (for characters with two skills of one kind).